eckit.geo and mir as a backend for gridefix-process

Assessment of grid, search and regridding support for POI interpolation and altitude correction

15 October 2026

1. Summary

gridefix-process (MeteoSwiss) interpolates gridded NWP output to points of interest (POIs) and to other grids. Its most specific algorithm corrects temperature from model orography to POI altitude using a gradient fitted locally on the k nearest grid points. This document assesses whether the eckit.geo infrastructure (Grid, Search, Projection) and mir (interpolation, weight matrices) can support these workflows as a backend, and what would need to change.

Grid and C++ coverage is adequate. eckit.geo already knows the relevant grids: ICON-CH1 and ICON-CH2 by name and uuid, Swiss LV95/LV03 through PROJ, regular x/y grids on any PROJ projection, and unstructured grids built from latitude/longitude arrays. The k-d tree core (3D Euclidean, cached by grid uid, with memory, mapped and shared-memory loaders) is suitable.

The Python API is not adequate yet. gridefix works on whole arrays, while `eckit.geo.Search` answers one point per call from Python and returns lists of tuples. This alone blocks adoption. A few other gaps (search over arbitrary point sets, masks, height-aware search, numpy coordinate output, construction from CF metadata) follow.

The altitude correction can be restructured. The local sounding fit is linear in the data values, so it can be precomputed as a sparse weight matrix and applied to every field, time step and ensemble member. The fit itself should stay in gridefix; eckit/mir would provide neighbour search, geometry and matrix handling around it.

2. Scope

Reviewed code:

- gridefix-process: `gridefix_process/grid_interp.py` (interp2poi, interp2grid and the interp_* methods), `grid_interp_utils.py`, `data_model.py` (grid and CRS detection), `optimization.py` (memoisation).
- eckit: `src/eckit/geo/Search.{h,cc}`, `search/KDTree.h`, `Grid.h`, figures and projections, `share/eckit/geo/*.yaml`, and the Cython bindings in `python/eckit/src/_eckit/`.
- mir: Python bindings (`Interpolation`, `Job`, `ArrayInput`/`ArrayOutput`, `WeightMatrix`) and the k-NN distance and pick methods.

No benchmarks were run. Statements about performance follow from the structure of the code.

3. Current gridefix approach

All non-regular methods follow the same pattern: flatten the grid, reproject grid and POI coordinates (to LV95 when altitude is involved, so that distances are in metres), build a `scipy.spatial.KDTree`, query the k nearest neighbours for all POIs at once (`tree.query(points, k, workers=-1)`), and combine neighbour values in numpy. Results are memoised on hashes of the raw coordinate bytes.

For the altitude-aware methods:

- **temp**: 3D k-NN on (E, N, z·z_fact) with z_fact = 25 and k = 36. For each POI a polynomial in altitude (degree 3 near the middle of the neighbour altitude range, degree 1 otherwise) is fitted by batched weighted QR and evaluated at the POI altitude. POIs far outside the neighbour altitude range use a fixed gradient. A `grad_only` variant fits degree 1 and clips the slope to [-0.01, 0.03] K/m.
- **temp_fixgrad / nearest_alt**: 2D k-NN with k = round(π·r_search²), then re-rank by horizontal distance + z_weight·max(|Δh| - 50 m, 0); fixgrad adds 0.006 K/m · Δh.
- **snowlim**: per trailing column, a nearest search restricted to non-NaN grid points.

4. Mapping to eckit.geo and mir

gridefix today	eckit / mir equivalent	Gap
`nn_lookup`: scipy KDTree, (n, k) distance and index arrays, multithreaded	`Search.search_knn(point, k)`	**One point per call, list output, no batch query, no threading.**
`memoize` keyed on `coords.tobytes()`	`SearchCache` keyed on grid uid and spec	None; the gridefix cache becomes unnecessary.
Reprojection to LV95 for metric distances (SWISS_CRS special cases)	(lon, lat) queries on a geocentric tree; distances are chord lengths in metres	None for 2D search; the LV95 detour can go.
3D search on (E, N, z·z_fact)	C++ `Search(std::vector<PointXYZ>)`	**Not exposed in Python; no height-aware mode.**
Dropping all-NaN grid points; per-column trees in snowlim	Search built from a whole Grid only	**No search restricted by a mask or index subset.**
`get_grid_crs`: CF grid_mapping, crs attribute, pyproj	Grid spec with `projection: {type: proj, source: ...}`	**No constructor from CF/xarray metadata.**
`scipy.griddata` (linear, cubic); `coarsen().mean()` before downsampling	mir linear, structured-bilinear/bicubic, grid-box-average, k-nn with distance weightings; `WeightMatrix.from_regrid`	**No helper to apply a weight matrix with missing values; no height-aware k-NN in mir.**
Flattened coordinates as numpy arrays	`to_points()` returns numpy; `to_latlons()`, `to_xyz()`, `x()`/`y()`/`lon()`/`lat()` return lists	**Lists are slow at ICON-CH1 size (about 1.1 million cells).**

5. The altitude correction is linear in the data

In the sounding fit, the weighted Vandermonde matrix depends only on neighbour orography and the weights only on neighbour distances. With QR factors Q and R of that matrix, the coefficients are $R^{-1}Q^{T}\mathrm{diag}(w)\cdot y$, and the value at the POI is $v_{poi}^{T}R^{-1}Q^{T}\mathrm{diag}(w)\cdot y$: a fixed linear combination of the k neighbour values y. The geometry does not change between fields, so:

- **Full sounding (degree 1 or 3)** is a sparse matrix W (n_poi × n_grid, k non-zeros per row). The rank-deficient fallback to degree 1 depends only on geometry and fits in the same matrix.
- **Fixed gradient** is affine: W·y + b.
- **grad_only** is y_0 + clip(G·y)·Δh with a sparse slope operator G: linear except for the clip.
- **idw, nearest, nearest_alt, upscaled with mean** are plain sparse matrices.

gridefix could therefore build W once per (grid, POIs, parameters) and compute `W @ data.reshape(n_grid, -1)` for every variable, lead time and member. The chunked per-POI QR loop over trailing dimensions disappears, and W can be cached on disk in the same way as mir interpolation matrices. The limitation is data-dependent masking (NaN patterns that vary per field); static masks such as land fraction are fine.

6. Required changes

Must have (eckit Python bindings)

1. **Batch queries.** `search.knn(lon, lat, k)` returning numpy (distance[n, k], index[n, k]); equivalent nn and radius queries (radius results as CSR-style offsets). Release the GIL and parallelise in C++.
2. **Concurrent queries.** Every query takes `shared_->mutex` in `Search.cc` because queries update k-d tree statistics, which serialises all queries and would cancel any gain from threading. Queries need a lock-free const path, for example per-thread or disabled statistics.
3. **Search over arbitrary points.** Expose `Search(std::vector<PointXYZ>)` or `Search(lon, lat)`, and accept `mask=` or `indices=` while returning indices in the original grid numbering.

4 **numpy output** for `to_latlons`, `to_xyz`, `x`, `y`, `lon`, `lat`.

Should have

5 **Height-aware search.** Map (lon, lat, h) to geocentric XYZ with height s·h; `convertSphericalToCartesian(..., height)` already exists. For s = 25 the horizontal distortion between neighbours is about 25 · 2500 m / 6371 km ≈ 1%, negligible for neighbour selection. It is not suitable for z_weight = 200 or 500, but nearest_alt and temp_fixgrad are a 2D k-NN followed by a re-ranking, which is a few lines of numpy once batch queries exist.

6 `Grid.from_cf(ds)` / `from_xarray`**.** Regular x/y with 1D coordinates and grid_mapping to regular_xy with a PROJ projection; latitude/longitude dimensions to regular_ll; ICON by `uuidOfHGrid`; anything else to an unstructured grid from 2D latitude/longitude. This would replace most of the CRS handling in `data_model.py`.

7 `WeightMatrix.apply(values, missing=nan)` **in mir.** Renormalise rows that touch missing values and keep trailing dimensions and dtype. interp2grid then becomes `from_regrid` followed by apply; grid-box-average replaces the slicing and coarsening helpers, and linear replaces griddata.

Nice to have (mir C++)

8 **Height-aware k-NN distance in mir** (for example nearest-orography), modelled on NearestLSM with input and output orography fields instead of land-sea masks. This would make nearest_alt and fixgrad regridding fully native in mir. The sounding fit is too specific for mir and should stay in gridefix as a weight builder.

7. Resulting client code

Sketch using the proposed API (none of these calls exist yet):

```
grid = eckit.geo.Grid.from_cf(ds)                    # or Grid("ICON-CH1")
search = grid.search(height=orog, vertical_scale=25)  # cached by uid + spec
dist, idx = search.knn(poi.lon, poi.lat, k=36, height=poi.alt)  # numpy (n, k)

W = sounding_weights(idx, dist, orog, poi.alt, deg=3)  # gridefix: scipy CSR, cacheable
out = W @ ds[var].values.reshape(grid.size(), -1)
```

Removable from gridefix:

- `nn_lookup`, `nn_lookup_cached`, `nn_lookup_regular` and the memoisation keys
- `_reproject_grid*`, `_convert_poi` and the SWISS_CRS logic
- `slice_boundingbox` and `_get_sub_domain` (only there to keep scipy tree builds small)
- `slice_coarsen`, `_get_coarsen_ratio` and the `griddata` calls

Stays in gridefix: the xarray glue, the sounding fit (as a weight builder) and the snow-limit rules.

8. Notes

- Distances: eckit returns chord lengths in metres on the grid's figure; gridefix uses planar LV95 metres. At neighbourhood scale the two agree closely, so IDW weights and the 40 km snow-limit cut-off keep their meaning.
- gridefix documentation: the `interp_temp` docstring gives a default z_fact of 50, the code uses 25; it describes the fixed-gradient fallback as 300 m above / 500 m below the neighbour range, the code uses +200 m / -600 m.
- Performance of eckit versus scipy k-NN has not been measured; it should be benchmarked once batch queries exist.